Polly and Resilience Patterns - Complete Guide Using Your Project

Part 1: The Problem Polly Solves

Imagine you're calling a friend on the phone:

Without Polly:

```
You call friend → No answer → You give up immediately
Result: You never get through!
```

With Polly:

```
You call friend → No answer → Wait 10 seconds → Call again
→ No answer → Wait 10 seconds → Call again
→ No answer → Wait 10 seconds → Call again
→ Friend answers! Success!
```

In Microservices:

- Services call each other over the network
- Network can fail (slow, timeout, connection lost)
- Databases can be temporarily unavailable
- External APIs can have issues

Without Polly: One failure = entire request fails **With Polly**: Automatically retry and handle failures gracefully

Part 2: Where Polly is Used in Your Project

Location 1: Database Migration Retry (Ordering Service)

File: Ordering.API/Entensions/DbExtension.cs (lines 21-28)

```
// retry strategy
var retry = Policy.Handle<SqlException>()
    .WaitAndRetry(
    retryCount: 5,
    sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
    onRetry: (exception, span, count) =>
    {
        logger.LogError($"Retrying because of {exception} {span}");
    });
retry.Execute(() => CallSeeder(seeder, context, services));
```

What this does:

- Policy.Handle() - Only retry if SQL exception occurs
- `retryCount: 5` - Retry up to 5 times
- `TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))` - Exponential backoff:
    - Retry 1: Wait 2 seconds (2^1)
    - Retry 2: Wait 4 seconds (2^2)
    - Retry 3: Wait 8 seconds (2^3)
    - Retry 4: Wait 16 seconds (2^4)
    - Retry 5: Wait 32 seconds (2^5)
- `onRetry` - Log each retry attempt
- `retry.Execute()` - Run the actual operation with retry logic

Why this is needed: When the application starts, SQL Server might not be ready yet (especially in Docker). This retries the migration instead of failing immediately.

Location 2: MassTransit Retry (Ordering & Payment Services)

File: Ordering.API/Program.cs (lines 36, 40, 45, 50)

```
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```

File: Payment/Program.cs (line 20)

```
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```

What this does:

- MassTransit uses Polly internally for retry
- `Interval(5, TimeSpan.FromSeconds(10))` - Retry 5 times, wait 10 seconds between each retry
- Applied to RabbitMQ message consumption

Why this is needed: If RabbitMQ is temporarily unavailable or message processing fails, it will retry instead of losing the message.

Part 3: Resilience Patterns Step-by-Step

Pattern 1: Retry Pattern

Problem: Temporary failures (network glitch, database restart)

Solution: Automatically retry the operation

Your code example (DbExtension.cs):

```
var retry = Policy.Handle<SqlException>()
    .WaitAndRetry(
        retryCount: 5,
        sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(Math.Pow(2, retryAttempt))
    );
```

How it works:

```
Attempt 1: Fails → Wait 2 seconds
Attempt 2: Fails → Wait 4 seconds
Attempt 3: Fails → Wait 8 seconds
```

```
Attempt 4: Fails → Wait 16 seconds
Attempt 5: Fails → Give up (throw exception)
```

Exponential Backoff: Each retry waits longer than the previous one. This prevents overwhelming the system when it's already struggling.

Pattern 2: Circuit Breaker Pattern

Problem: If a service is down, retrying won't help - it just wastes time

Solution: Stop trying for a while, then try again later

Visual:

```
Closed (Normal) → Open (Stop trying) → Half-Open (Test) → Closed (Normal)
```

Example (NOT in your code, but this is how it would look):

```
var circuitBreaker = Policy
    .Handle<Exception>()
    .CircuitBreaker(
        exceptionsAllowedBeforeBreaking: 3,
        durationOfBreak: TimeSpan.FromSeconds(30)
    );
```

How it works:

- After 3 failures, stop trying for 30 seconds
- After 30 seconds, try once (Half-Open)
- If success, go back to normal (Closed)
- If failure, stay open for another 30 seconds

Your project doesn't use Circuit Breaker yet - you could add it for external service calls!

Pattern 3: Timeout Pattern

Problem: Operation hangs forever (slow database, network timeout)

Solution: Give up after a specific time

Example (NOT in your code):

```
var timeout = Policy
    .Timeout(TimeSpan.FromSeconds(30));
```

How it works:

- If operation takes more than 30 seconds, throw TimeoutException
- Prevents your application from hanging

Pattern 4: Fallback Pattern

Problem: Operation fails, but you need to return something

Solution: Return a default value or cached data

Example (NOT in your code):

```
var fallback = Policy
    .Handle<Exception>()
    .Fallback("Default Value");
```

How it works:

- If operation fails, return "Default Value" instead of throwing exception
- Good for: returning cached data, default response, or empty list

Part 4: Polly Package Installation

Your file: Ordering.API/Ordering.API.csproj (line 21)

```
<PackageReference Include="Polly" Version="8.6.5" />
```

To install:

```
dotnet add package Polly
```

Part 5: Complete Example - Adding Polly to Basket Service

Let's say you want to add retry to the Discount gRPC call in Basket service:

Without Polly (current code in CreateShoppingCartHandler.cs):

```
var coupon = await _discountGrpcService.GetDiscount(item.ProductName);
```

With Polly (how to add it):

```
using Polly;

// In Program.cs - register Polly
builder.Services.AddHttpClient();

// In Handler - use Polly
var retryPolicy = Policy
    .Handle<Grpc.Core.RpcException>()
    .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(2));

var coupon = await retryPolicy.ExecuteAsync(async () =>
{
    return await _discountGrpcService.GetDiscount(item.ProductName);
});
```

Part 6: MassTransit Retry (Your Current Implementation)

Your code (Ordering.API/Program.cs lines 36, 40, 45, 50):

```
cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));
```

This is Polly retry under the hood! MassTransit uses Polly for its retry mechanism.

What it does:

- When consuming messages from RabbitMQ
- If processing fails, retry up to 5 times
- Wait 10 seconds between each retry
- After 5 failures, move message to dead-letter queue

Why this is critical:

- Messages shouldn't be lost
- Temporary failures (database timeout, network glitch) shouldn't cause message loss
- After retries, message goes to dead-letter queue for manual inspection

Part 7: Summary of Your Polly Usage

Location	Pattern	Purpose
DbExtension.cs	Retry with Exponential Backoff	Database migration retry on startup
Ordering.API/Program.cs	Retry (via MassTransit)	RabbitMQ message consumption retry
Payment/Program.cs	Retry (via MassTransit)	RabbitMQ message consumption retry

Part 8: Why You Need Polly in Microservices

Microservices Reality:

- Services call each other over network (unreliable)
- Databases can be temporarily unavailable
- External APIs can have rate limits or downtime
- Network glitches happen

Without Polly:

- One failure = entire request fails
- Bad user experience

- Lost messages
- Cascading failures (one service down brings down others)

With Polly:

- Automatic retry on temporary failures
- Circuit breaker prevents overwhelming struggling services
- Timeout prevents hanging
- Fallback provides graceful degradation
- Better user experience
- System stays resilient

I've completed the comprehensive guide on Polly and Resilience Patterns using your project code.

Summary of Your Polly Usage

1. Database Migration Retry (Ordering.API/Entensions/DbExtension.cs):

- Uses Polly's `WaitAndRetry` with exponential backoff
- Retries SQL database migration up to 5 times
- Waits 2, 4, 8, 16, 32 seconds between retries
- Prevents startup failure if SQL Server isn't ready

2. MassTransit Message Retry (Ordering.API/Program.cs and Payment/Program.cs):

- Uses MassTransit's built-in Polly retry
- Retries RabbitMQ message consumption 5 times
- Waits 10 seconds between each retry
- Prevents message loss on temporary failures

Key Resilience Patterns Explained

- **Retry Pattern**: Automatically retry on temporary failures (you're using this)
- **Circuit Breaker**: Stop trying when service is down (not in your code yet)
- **Timeout Pattern**: Give up after specific time (not in your code yet)
- **Fallback Pattern**: Return default value on failure (not in your code yet)

Why Polly is Critical for Microservices

Your microservices communicate over unreliable networks and depend on external services (databases, RabbitMQ, gRPC). Without Polly, a single temporary failure would cause the entire request to fail. With Polly, your system automatically handles these failures gracefully, providing better reliability and user experience.

Your current implementation is solid for database startup and message consumption. You could enhance it further by adding Circuit Breaker for external service calls (like the Discount gRPC call in Basket service).

Circuit Breaker in Polly - Complete Guide

Part 1: Circuit Breaker Status in Your Project

Answer: You are NOT using Circuit Breaker in your project yet.

Your current Polly usage:

- ✅ **Retry Pattern** - Used in DbExtension.cs and MassTransit
- ❌ **Circuit Breaker Pattern** - NOT implemented anywhere

Part 2: What is Circuit Breaker? (The Concept)

Imagine a light switch in your house:

```
Switch OFF (Circuit Open)  →  No electricity flows
Switch ON (Circuit Closed) →  Electricity flows normally
```

In Software:

- **Closed (Normal)**: Everything works, requests go through
- **Open (Stop)**: Too many failures, stop trying for a while
- **Half-Open (Test)**: Try one request to see if service recovered

Real-world example:

```
You call a friend who is busy:
Call 1: No answer
Call 2: No answer
Call 3: No answer
→ You stop calling for 30 minutes (Circuit Open)
→ After 30 minutes, you try once (Half-Open)
→ If they answer: Go back to normal calling (Closed)
→ If no answer: Wait another 30 minutes (Stay Open)
```

Why this is better than just retrying:

- **Retry**: Keep trying even if service is down (wastes time/resources)
- **Circuit Breaker**: Stop trying when service is clearly down (saves time/resources)

Part 3: Where to Implement Circuit Breaker in Your Project

Best place: Basket service's gRPC call to Discount service

Current code (Basket.Application/Handlers/CreateShoppingCartHandler.cs lines 24-28):

```
foreach (var item in request.items)
{
    var coupon = await
_discountGrpcService.GetDiscount(item.ProductName);
    item.Price = item.Price - coupon.Amount;
}
```

Problem: If Discount service is down, this will keep failing and retrying (if you have retry), wasting time and resources.

Solution: Add Circuit Breaker to stop trying after a few failures.

Part 4: Step-by-Step Implementation Guide

Step 1: Add Polly NuGet Package (if not already there)

File: Basket.Application/Basket.Application.csproj

```
<ItemGroup>
    <PackageReference Include="Polly" Version="8.6.5" />
</ItemGroup>
```

Install command:

```
cd Services/Basket/Basket.Application
dotnet add package Polly
```

Step 2: Create a Policy Service (Best Practice)

Create a new file: `Basket.Application/Services/DiscountPolicy.cs`

```
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
    public static class DiscountPolicy
    {
        public static IAsyncPolicy<Grpc.Core.RpcException>
GetCircuitBreakerPolicy()
        {
            return Policy
                .Handle<RpcException>()  // Handle gRPC exceptions
                .CircuitBreakerAsync(
                    exceptionsAllowedBeforeBreaking: 3,  // Break
after 3 failures
                    durationOfBreak: TimeSpan.FromSeconds(30)  // Stay
open for 30 seconds
                );
        }
```

```
        public static IAsyncPolicy<Grpc.Core.RpcException>
GetRetryPolicy()
        {
            return Policy
                .Handle<RpcException>()
                .WaitAndRetryAsync(
                    retryCount: 3,
                    sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(2)
                );
        }
    }
}
```

Step 3: Combine Retry + Circuit Breaker

Update the policy service:

```
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
    public static class DiscountPolicy
    {
        public static IAsyncPolicy<Grpc.Core.RpcException>
GetDiscountPolicy()
        {
            // Retry first, then Circuit Breaker
            return Policy
                .Handle<RpcException>()
                .WaitAndRetryAsync(3, retryAttempt =>
TimeSpan.FromSeconds(2))
                .WrapAsync(
                    Policy
                        .Handle<RpcException>()
                        .CircuitBreakerAsync(
```

```
                        exceptionsAllowedBeforeBreaking: 3,
                        durationOfBreak: TimeSpan.FromSeconds(30)
                    )
                );
        }
    }
}
```

What this does:

- First: Retry up to 3 times with 2-second delay
- Then: If still failing after retries, open circuit for 30 seconds
- Wrap: Combines both policies (Retry inside Circuit Breaker)

Step 4: Use in Handler

File: Basket.Application/Handlers/CreateShoppingCartHandler.cs

```
using Basket.Application.Commands;
using Basket.Application.GrpcService;
using Basket.Application.Mappers;
using Basket.Application.Responses;
using Basket.Application.Services;  // Add this
using Basket.Core.Repositories;
using MediatR;
using Polly;  // Add this

namespace Basket.Application.Handlers
{
    public class CreateShoppingCartHandler :
IRequestHandler<CreateShoppingCartCommand, ShoppingCartResponse>
    {
        private readonly IBasketRepository _basketRepository;
        private readonly DiscountGrpcService _discountGrpcService;
        private readonly IAsyncPolicy<RpcException> _discountPolicy;
// Add this

        public CreateShoppingCartHandler(
            IBasketRepository basketRepository,
```

```
            DiscountGrpcService discountGrpcService)
        {
            _basketRepository = basketRepository;
            _discountGrpcService = discountGrpcService;
            _discountPolicy = DiscountPolicy.GetDiscountPolicy();  //
Initialize policy
        }

        public async Task<ShoppingCartResponse>
Handle(CreateShoppingCartCommand request, CancellationToken
cancellationToken)
        {
            // Apply discounts using GRPC call with Circuit Breaker
            foreach (var item in request.items)
            {
                var coupon = await _discountPolicy.ExecuteAsync(async
() =>
                {
                    return await
_discountGrpcService.GetDiscount(item.ProductName);
                });
                item.Price = item.Price - coupon.Amount;
            }

            // Convert command to domain entity
            var shoppingCartEntity = request.ToShoppingCartEntity();

            // Save to redis
            var updateCart = await
_basketRepository.UpsertBasket(shoppingCartEntity);

            // Convert back to response
            return updateCart.ToShoppingCartResponse();
        }
    }
}
```

Step 5: Add Fallback (Optional but Recommended)

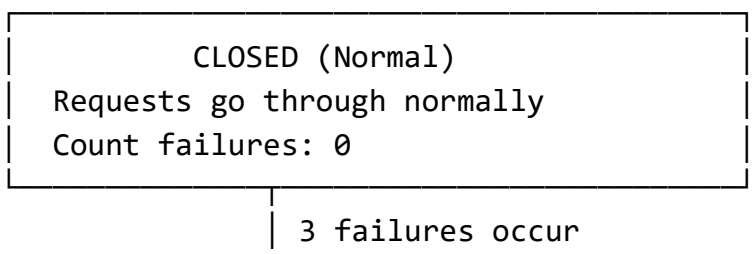
When circuit is open, return default discount (0):

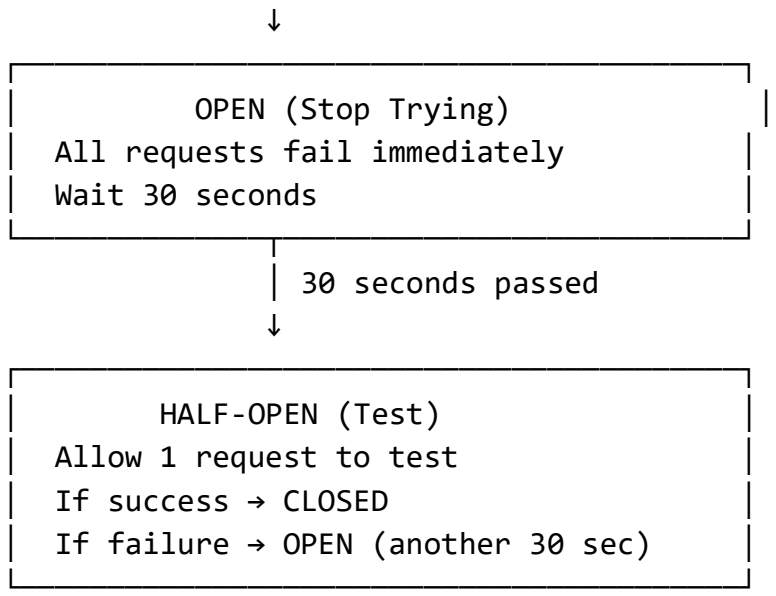
```
public static IAsyncPolicy<Grpc.Core.RpcException>
GetDiscountPolicyWithFallback()
{
    var circuitBreaker = Policy
        .Handle<RpcException>()
        .CircuitBreakerAsync(
            exceptionsAllowedBeforeBreaking: 3,
            durationOfBreak: TimeSpan.FromSeconds(30)
        );

    var fallback = Policy
        .Handle<RpcException>()
        .FallbackAsync(async (exception) =>
        {
            // Return default discount when circuit is open
            return new Discount.Grpc.Protos.CouponModel
            {
                Amount = 0,
                ProductName = "default",
                Description = "Discount service unavailable"
            };
        });

    return fallback.WrapAsync(circuitBreaker);
}
```

Step 6: Circuit Breaker States Explained





Step 7: Monitor Circuit Breaker State (Advanced)

Add logging to track state changes:

```
public static IAsyncPolicy<Grpc.Core.RpcException>
GetCircuitBreakerWithLogging()
{
    return Policy
        .Handle<RpcException>()
        .CircuitBreakerAsync(
            exceptionsAllowedBeforeBreaking: 3,
            durationOfBreak: TimeSpan.FromSeconds(30),
            onBreak: (exception, breakDelay) =>
            {
                Console.WriteLine($"Circuit OPEN due to:
{exception.Message}. Will stay open for {breakDelay.TotalSeconds}
seconds");
            },
            onReset: () =>
            {
                Console.WriteLine("Circuit CLOSED - service
recovered");
            },
            onHalfOpen: () =>
```

```
            {
                Console.WriteLine("Circuit HALF-OPEN - testing
service");
            }
        );
}
```

Part 5: Complete Working Example

File: Basket.Application/Services/DiscountPolicy.cs

```
using Polly;
using Grpc.Core;

namespace Basket.Application.Services
{
    public static class DiscountPolicy
    {
        public static IAsyncPolicy<Grpc.Core.RpcException>
GetDiscountPolicy()
        {
            // Retry policy: Try 3 times with 2-second delay
            var retryPolicy = Policy
                .Handle<RpcException>()
                .WaitAndRetryAsync(
                    retryCount: 3,
                    sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(2),
                    onRetry: (exception, timeSpan, retryCount,
context) =>
                    {
                        Console.WriteLine($"Retry {retryCount} after
{timeSpan.TotalSeconds}s due to: {exception.Message}");
                    }
                );

            // Circuit Breaker: Open after 3 failures, stay open for
30 seconds
```

```
            var circuitBreakerPolicy = Policy
                .Handle<RpcException>()
                .CircuitBreakerAsync(
                    exceptionsAllowedBeforeBreaking: 3,
                    durationOfBreak: TimeSpan.FromSeconds(30),
                    onBreak: (exception, breakDelay) =>
                    {
                        Console.WriteLine($"Circuit OPEN for
{breakDelay.TotalSeconds}s due to: {exception.Message}");
                    },
                    onReset: () =>
                    {
                        Console.WriteLine("Circuit CLOSED - service
recovered");
                    },
                    onHalfOpen: () =>
                    {
                        Console.WriteLine("Circuit HALF-OPEN - testing
service");
                    }
                );

            // Combine: Retry first, then Circuit Breaker
            return Policy.WrapAsync(retryPolicy,
circuitBreakerPolicy);
        }
    }
}
```

File: Basket.Application/Handlers/CreateShoppingCartHandler.cs (updated)

```
public async Task<ShoppingCartResponse>
Handle(CreateShoppingCartCommand request, CancellationToken
cancellationToken)
{
    var policy = DiscountPolicy.GetDiscountPolicy();

    foreach (var item in request.items)
    {
```

```
        try
        {
            var coupon = await policy.ExecuteAsync(async () =>
            {
                return await
_discountGrpcService.GetDiscount(item.ProductName);
            });
            item.Price = item.Price - coupon.Amount;
        }
        catch (BrokenCircuitException)
        {
            // Circuit is open - use default discount (0)
            Console.WriteLine("Circuit is OPEN - using default
discount");
            item.Price = item.Price;  // No discount
        }
        catch (RpcException ex)
        {
            // Other gRPC errors
            Console.WriteLine($"gRPC error: {ex.Message}");
            item.Price = item.Price;  // No discount
        }
    }

    var shoppingCartEntity = request.ToShoppingCartEntity();
    var updateCart = await
_basketRepository.UpsertBasket(shoppingCartEntity);
    return updateCart.ToShoppingCartResponse();
}
```

Summary

Circuit Breaker Status: NOT implemented in your project yet.

Your Current Polly Usage:

- ✅ Retry Pattern (in DbExtension.cs and MassTransit)
- ❌ Circuit Breaker Pattern (not implemented)

Best Place to Add Circuit Breaker: Basket service's gRPC call to Discount service (CreateShoppingCartHandler.cs)

Implementation Steps:

1. Add Polly package to Basket.Application
2. Create `DiscountPolicy.cs` with Circuit Breaker + Retry
3. Update CreateShoppingCartHandler.cs to use the policy
4. Handle `BrokenCircuitException` when circuit is open

Benefits:

- Stops calling Discount service when it's clearly down
- Saves time and resources
- Prevents cascading failures
- Better user experience (uses default discount instead of failing)

Would you like me to implement the Circuit Breaker in your Basket service by creating the `DiscountPolicy.cs` file and updating the CreateShoppingCartHandler.cs?